

Gestiona dotfiles fácilmente con GNU Stow

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

This tutorial provides a step-by-step guide to managing dotfiles using GNU Stow, a tool that leverages symbolic links to centralize and synchronize configuration files across Unix-like systems (Linux, macOS, WSL). It explains the importance of dotfiles, such as `.bashrc` and `.gitconfig`, in customizing user environments and highlights the inefficiencies of manual management. The guide details installing GNU Stow, creating a dotfiles repository, linking configurations, and automating the process with a bash script. Advanced tips include handling conflicts, platform-specific setups, and alternatives like Chezmoi and YADM. This resource is designed for developers seeking efficient, portable configuration management.

Palabras Claves: Dotfiles, GNU Stow, Symbolic links, Configuration management, Git integration

Tabla de contenidos

Introduction	6
1 Instalación	8
1.1 Linux	8
1.2 macOS	8
1.3 Desde Fuente	8
1.4 Verificar Instalación	9
2 Conceptos Fundamentales	9
2.1 Terminología Clave	9
2.2 1. Package (Paquete)	9
2.3 2. Target Directory (Directorio Objetivo)	9
2.4 3. Stow Directory (Directorio Stow)	10
2.5 4. Installation Image (Imagen de Instalación)	10
2.6 5. Symlink (Enlace Simbólico)	10

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, redacción

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Email: elmer.achalma.09@unsch.edu.pe

2.7	Jerarquía de Directorios	10
3	Sintaxis y Comandos	11
3.1	Sintaxis Básica	11
3.2	Acciones Principales	11
3.3	1. Stow (Instalar)	11
3.4	2. Delete (Desinstalar)	11
3.5	3. Restow (Reinstalar)	11
3.6	Opciones de Directorio	12
3.7	-d / --dir (Stow Directory)	12
3.8	-t / --target (Target Directory)	12
3.9	Opciones de Simulación y Verbosidad	12
3.10	-n / --no / --simulate (Dry Run)	12
3.11	-v / --verbose (Verbosidad)	13
3.12	Opciones Avanzadas	13
3.13	--ignore (Ignorar Archivos)	13
3.14	--defer (Diferir)	13
3.15	--override (Sobrescribir)	13
3.16	--dotfiles (Modo Dotfiles)	13
3.17	--no-folding (Sin Tree Folding)	14
3.18	--adopt (Adoptar Archivos)	14
3.19	Combinando Operaciones	14
4	Estructura de Directorios	14
4.1	Estructura Recomendada para Dotfiles	14
4.2	Principios de Organización	15
4.3	1. Un Directorio = Un Paquete	15
4.4	2. Replicar Estructura del HOME	15
4.5	3. Agrupar Lógicamente	16
4.6	Ejemplos de Estructuras	16
4.7	Estructura Simple	16
4.8	Estructura Compleja	17
5	Instalación de Paquetes	18
5.1	Proceso de Instalación	18
5.2	1. Tree Folding (Plegado de Árbol)	18
5.3	2. Tree Unfolding (Desplegado de Árbol)	19
5.4	Instalación Básica	19
5.5	Instalación con Verificación	20
5.6	Instalación Selectiva	20
6	Desinstalación de Paquetes	20
6.1	Proceso de Desinstalación	20
6.2	1. Eliminación de Symlinks	20
6.3	2. Eliminación de Directorios Vacíos	20
6.4	3. Tree Refolding (Re-plegado)	21
6.5	Desinstalación Básica	21
6.6	Desinstalación con Verificación	21
6.7	Desinstalación Parcial	22

7	Reinstalación de Paquetes	22
7.1	Comando Restow	22
7.2	Cuándo Usar Restow	22
7.3	Restow vs Delete + Stow	22
8	Gestión de Dotfiles	23
8.1	Setup Inicial	23
8.2	1. Crear Estructura	23
8.3	2. Mover Configuraciones Existentes	23
8.4	3. Usar --adopt (Con Precaución)	24
8.5	Workflow Diario	24
8.6	Editar Configuraciones	24
8.7	Agregar Nueva Aplicación	24
8.8	Sincronizar con Git	25
8.9	Manejo de Archivos Sensibles	25
8.10	Estrategia 1: .gitignore	25
8.11	Estrategia 2: Archivos Template	25
8.12	Estrategia 3: Encriptación	26
8.13	Estructura para Múltiples Hosts	26
9	Ignore Lists	27
9.1	Tipos de Ignore Lists	27
9.2	1. Built-in (Predeterminado)	27
9.3	2. Global Ignore List	27
9.4	3. Package-Local Ignore List	28
9.5	Sintaxis de Ignore Lists	28
9.6	Reglas de Matching	28
9.7	Ejemplos Prácticos	29
9.8	Precedencia de Ignore Lists	29
9.9	Opción --ignore en CLI	29
10	Opciones Avanzadas	30
10.1	Tree Folding Control	30
10.2	--no-folding	30
10.3	Adopt Mode	30
10.4	--adopt	30
10.5	Defer y Override	31
10.6	--defer	31
10.7	--override	31
10.8	Dotfiles Mode	32
10.9	--dotfiles	32
10.10	Multiple Stow Directories	32
11	Casos de Uso Prácticos	33
11.1	Caso 1: Setup en Nueva Máquina	33
11.2	Caso 2: Actualizar Configuraciones	34
11.3	Caso 3: Probar Nueva Configuración	34
11.4	Caso 4: Compartir Configuración entre Usuarios	35
11.5	Caso 5: Configuración Por Entorno	35

12 Tu Repositorio .dotfiles	36
12.1 Análisis de tu Estructura Actual	36
12.2 Implementación de Stow en tu Repo	37
12.3 1. Script install.sh Mejorado	37
12.4 2. Script para Desinstalar	39
12.5 3. Reorganizar Paquetes Problemáticos	40
12.6 4. .stowrc para tu Repo	40
12.7 5. .stow-local-ignore por Paquete	41
12.8 6. Script de Verificación	42
12.9 7. Actualizar .gitignore	42
12.10 8. Comandos Útiles para tu Repo	43
13 Workflows Completos	44
13.1 Workflow 1: Configuración Inicial	44
13.2 Workflow 2: Día a Día	44
13.3 Workflow 3: Nueva Máquina	45
13.4 Workflow 4: Experimentar	45
13.5 Workflow 5: Actualización Limpia	46
14 Integración con Git	46
14.1 Estructura de Repositorio	46
14.2 .gitignore Completo	47
14.3 Commits Best Practices	48
14.4 Branches Strategy	49
14.5 Tags para Versiones	49
14.6 Submodules para Plugins	49
14.7 GitHub Actions para Validación	50
15 Troubleshooting	50
15.1 Problema 1: Conflictos al Stow	50
15.2 Problema 2: Symlinks Rotos	51
15.3 Problema 3: Directorio No Vacío	52
15.4 Problema 4: Tree Folding Inesperado	52
15.5 Problema 5: Permiso Denegado	52
15.6 Problema 6: Stow No Encuentra Paquete	53
15.7 Problema 7: .stowrc No Se Aplica	53
15.8 Problema 8: Stow Muy Lento	54
16 Scripts de Automatización	55
16.1 Script 1: install.sh Completo	55
16.2 Script 2: update.sh	60
16.3 Script 3: check.sh	61
16.4 Script 4: clean.sh	62
17 Best Practices	62
17.1 1. Organización de Paquetes	62
17.2 2. Uso de Ignore Lists	63
17.3 3. Commits y Mensajes	63
17.4 4. Testing Antes de Commit	63

17.5	5. Backup Siempre	64
17.6	6. Documentación	64
17.7	7. Estructura Consistente	65
17.8	8. Versionado	65
18	Alternativas a Stow	65
18.1	1. yadm (Yet Another Dotfiles Manager)	65
18.2	2. chezmoi	66
18.3	3. dotbot	66
18.4	4. Bare Git Repository	67
18.5	Comparación	67
19	Conclusión	67
19.1	Resumen de Stow	67
19.2	Comandos Esenciales	68
19.3	Próximos Pasos	68
19.4	Recursos Adicionales	68
20	Publicaciones Similares	69

Gestiona dotfiles fácilmente con GNU Stow

¿Alguna vez has perdido horas configurando tu terminal o editor tras cambiar de computadora? Los **dotfiles**, esos archivos ocultos como `.bashrc` o `.gitconfig`, guardan tus personalizaciones, pero gestionarlos a mano es un caos. **GNU Stow** simplifica todo: organiza tus configuraciones en un repositorio central y usa **enlaces simbólicos** para sincronizarlas en minutos.

¿Qué es Dotfiles?

Los **dotfiles** son archivos ocultos en sistemas Unix (Linux, macOS) que empiezan con un punto (ej., `.zshrc`, `.gitconfig`, `.config/nvim`). Almacenan configuraciones personalizadas para tu terminal, editor de código o gestor de ventanas. Por ejemplo, `.bashrc` define alias y variables de entorno, mientras que `.vimrc` ajusta tu editor Vim. Estos archivos son el corazón de tu flujo de trabajo, ya que personalizan tus herramientas favoritas.

Tener **dotfiles** bien organizados te ahorra horas al replicar tu entorno en nuevas máquinas. Imagina configurar tu shell o editor desde cero tras reinstalar tu sistema: ¡es tedioso! Con una gestión adecuada, puedes clonar tus configuraciones y tener todo listo rápidamente. Esto es importante para desarrolladores que trabajan en múltiples dispositivos o entornos como servidores y laptops.

Problemas de la Gestión Manual

Copiar **dotfiles** manualmente o usar scripts caseros es lento y arriesgado. Puedes sobrescribir archivos, olvidar configuraciones o perderlas en una reinstalación. Por ejemplo, mover `.zshrc` a otra máquina sin un sistema organizado puede causar errores si las versiones del software difieren. **GNU Stow** soluciona esto al centralizar tus archivos y crear **enlaces simbólicos** automáticamente, manteniendo todo sincronizado.

¿Qué es GNU Stow?

GNU Stow es un **gestor de granjas de enlaces simbólicos** (symlink farm manager) que permite administrar múltiples paquetes de software o conjuntos de archivos de configuración de manera organizada. Concepto principal:

```
Instalar cada paquete en su propio árbol de directorios
↓
Usar enlaces simbólicos para que aparezcan en un árbol común
↓
Administrar fácilmente cada paquete de forma independiente
```

Problema original:

```
# En /usr/local/man/man1 tenías:
a2p.1      # ¿De qué paquete es?
perl.1     # ¿Perl?
emacs.1    # ¿Emacs?
etags.1    # ¿Emacs también?

# Al desinstalar Perl... ¿qué archivos eliminar?
```

Solución con Stow:

```
# Cada paquete en su propio árbol:
/usr/local/stow/perl/
  bin/
    perl
    a2p
  man/
    man1/
      perl.1
      a2p.1

/usr/local/stow/emacs/
  bin/
    emacs
  man/
    man1/
      emacs.1

# Stow crea symlinks en /usr/local/ que apuntan a los paquetes
```

Gestión de Dotfiles

Aunque Stow fue diseñado para software, hoy en día su uso principal es **gestionar dot-files**:

Ventajas:

- Mantener dotfiles organizados por aplicación
- Sincronizar con Git
- Instalar/desinstalar configuraciones selectivamente
- Mantener backups sin perder estructura
- Compartir configuraciones entre máquinas
- Control de versiones granular

Comparación: Antes vs Después de Stow

Sin Stow:

```
~/.config/
  nvim/
  kitty/
  zsh/
  ...

# Problemas:
# - Difícil hacer backup selectivo
# - No hay organización por paquete
# - Complicado compartir entre máquinas
# - Sin control de versiones granular
```

Con Stow:

```
~/dotfiles/          # Stow directory
  nvim/              # Package
    .config/
      nvim/
kitty/              # Package
  .config/
    kitty/
zsh/                # Package
  .zshrc
  .zshenv

# Ventajas:
# - Cada aplicación es un "paquete"
# - Fácil stow/unstow selectivo
# - Git maneja cada paquete independientemente
# - Estructura clara y mantenible
```

1 Instalación

1.1 Linux

Ubuntu/Debian:

```
sudo apt update
sudo apt install stow
```

Arch Linux:

```
sudo pacman -S stow
```

Fedora/RHEL:

```
sudo dnf install stow
```

openSUSE:

```
sudo zypper install stow
```

1.2 macOS

```
# Con Homebrew
brew install stow

# O con MacPorts
sudo port install stow
```

1.3 Desde Fuente


```
# Descargar última versión
wget https://ftp.gnu.org/gnu/stow/stow-latest.tar.gz
tar -xzf stow-latest.tar.gz
cd stow-2.4.1

# Compilar e instalar
./configure
make
sudo make install
```

1.4 Verificar Instalación

```
# Verificar versión
stow --version
# GNU Stow version 2.4.1

# Ver ayuda
stow --help
```

2 Conceptos Fundamentales

2.1 Terminología Clave

2.2 1. Package (Paquete)

Una colección relacionada de archivos y directorios que administras como una unidad.

```
# Ejemplo: paquete "nvim"
nvim/
  .config/
    nvim/
      init.lua
      lua/
  .local/
    share/
      nvim/
```

2.3 2. Target Directory (Directorio Objetivo)

El directorio raíz donde quieres que aparezcan instalados tus paquetes.

```
# Para dotfiles, usualmente es:
Target: ~/ (tu HOME)

# Para software del sistema:
Target: /usr/local
```

2.4 3. Stow Directory (Directorio Stow)

El directorio raíz que contiene todos tus paquetes en subdirectorios separados.

```
# Para dotfiles:
Stow dir: ~/dotfiles/

# Para software:
Stow dir: /usr/local/stow/
```

2.5 4. Installation Image (Imagen de Instalación)

La estructura de archivos y directorios requerida por un paquete, relativa al target directory.

```
# El paquete "zsh" tiene esta imagen:
zsh/
  .zshrc          # → ~/.zshrc
  .zshenv         # → ~/.zshenv
  .config/
    zsh/          # → ~/.config/zsh/
    aliases.zsh
```

2.6 5. Symlink (Enlace Simbólico)

Un archivo especial que apunta a otro archivo o directorio.

```
# Ejemplo:
~/.zshrc -> ~/dotfiles/zsh/.zshrc
      ↑
    symlink
```

Tipos de symlinks:

- **Absoluto:** /home/user/dotfiles/zsh/.zshrc
- **Relativo:** ../dotfiles/zsh/.zshrc

Nota: Stow solo crea symlinks **relativos** dentro del target directory.

2.7 Jerarquía de Directorios

/home/user/ (target directory)

```
.zshrc
.config/
  nvim/      symlinks
  kitty/
```

↓

```
/home/user/dotfiles/ (stow dir)
```

```
zsh/      (package)
  .zshrc
nvim/     (package)
  .config/
    nvim/
kitty/   (package)
  .config/
    kitty/
```

3 Sintaxis y Comandos

3.1 Sintaxis Básica

```
stow [opciones] [flags de acción] paquete1 paquete2 ...
```

3.2 Acciones Principales

3.3 1. Stow (Instalar)

```
# Instalar un paquete
stow nombre-paquete

# Instalar múltiples paquetes
stow nvim zsh kitty

# Flag explícito (opcional)
stow -S nvim
stow --stow nvim
```

3.4 2. Delete (Desinstalar)

```
# Desinstalar un paquete
stow -D nvim
stow --delete nvim

# Desinstalar múltiples
stow -D nvim zsh kitty
```

3.5 3. Restow (Reinstalar)

```
# Unstow + Stow en una operación
stow -R nvim
stow --restow nvim

# Útil después de actualizar paquete
```

3.6 Opciones de Directorio

3.7 -d / --dir (Stow Directory)

```
# Especificar stow directory
stow -d ~/mis-dotfiles -t ~ nvim

# Default: directorio actual
```

3.8 -t / --target (Target Directory)

```
# Especificar target directory
stow -t /usr/local perl

# Default: padre del stow directory
```

Ejemplo completo:

```
# Estructura:
/opt/
  myapps/      # stow directory
    myapp/     # package
      bin/
        myapp

# Comando:
cd /opt/myapps
stow -t /usr/local myapp

# Resultado:
/usr/local/bin/myapp -> ../opt/myapps/myapp/bin/myapp
```

3.9 Opciones de Simulación y Verbosidad

3.10 -n / --no / --simulate (Dry Run)

```
# Mostrar qué haría sin hacer cambios
stow -n nvim
stow --simulate nvim

# Combinado con verbose
stow -nv nvim
```

3.11 -v / --verbose (Verbosidad)

```
# Niveles de verbosidad: 0-5
stow -v nvim          # verbose level 1
stow -vv nvim         # verbose level 2
stow --verbose=5 nvim # verbose level 5

# Nivel 0: silencioso (default)
# Nivel 1-2: operaciones principales
# Nivel 3-5: debug detallado
```

Ejemplo:

```
$ stow -nv zsh
WARNING! stowing zsh would cause conflicts:
  * existing target is neither a link nor a directory: .zshrc
All operations aborted.
```

3.12 Opciones Avanzadas

3.13 --ignore (Ignorar Archivos)

```
# Ignorar archivos que coincidan con regexp
stow --ignore='.*\.orig' --ignore='.*\.dist' nvim

# Múltiples patrones
stow --ignore='README.*' --ignore='.*~' nvim
```

3.14 --defer (Diferir)

```
# No sobrescribir si ya existe desde otro paquete
stow --defer=man --defer=info perl
```

3.15 --override (Sobrescribir)

```
# Forzar sobrescribir symlinks existentes
stow --override=man --override=info perl
```

3.16 --dotfiles (Modo Dotfiles)

```
# Transforma "dot-" en "."
# dot-bashrc → .bashrc
stow --dotfiles bash

# Ejemplo de paquete:
bash/
  dot-bashrc    # Se convierte en ~/.bashrc
```

3.17 --no-folding (Sin Tree Folding)

```
# Desactivar tree folding
stow --no-folding nvim

# Crea directorios en lugar de symlinks a directorios
```

3.18 --adopt (Adoptar Archivos)

```
# CUIDADO: Modifica el stow directory
# Mueve archivos del target al package

stow --adopt nvim

# Si ~/.config/nvim/init.lua existe:
# Lo mueve a ~/.dotfiles/nvim/.config/nvim/init.lua
# Luego crea el symlink
```

3.19 Combinando Operaciones

```
# Mezclar múltiples acciones
stow -D old-nvim -S new-nvim

# Orden de ejecución:
# 1. Unstow old-nvim
# 2. Stow new-nvim

# Múltiples paquetes, múltiples acciones
stow -S pkg1 pkg2 -D pkg3 pkg4 -S pkg5 -R pkg6
# Resultado: unstow pkg3,4,6 → stow pkg1,2,5,6
```

4 Estructura de Directorios

4.1 Estructura Recomendada para Dotfiles

```
~/dotfiles/                                # Stow directory
  git/                                     # Package
    .gitconfig
  zsh/                                     # Package
    .zshrc
    .zshenv
    .config/
      zsh/
        aliases.zsh
        functions.zsh
  nvim/                                    # Package
```

```

    .config/
      nvim/
        init.lua
        lua/
          plugins/
          config/
kitty/                                     # Package
  .config/
    kitty/
      kitty.conf
      themes/
tmux/                                     # Package
  .tmux.conf
  .config/
    tmux/
kde/                                     # Package
  .config/
    kdeglobals
    dolphinrc
    kwinrc

```

4.2 Principios de Organización

4.3 1. Un Directorio = Un Paquete

```

# Bien: un paquete por aplicación
nvim/
  .config/
    nvim/

# Mal: múltiples aplicaciones en un paquete
editors/
  .config/
    nvim/
    vim/
  .vimrc

```

4.4 2. Replicar Estructura del HOME

```

# El contenido del paquete debe replicar la estructura de ~/

# Ejemplo: archivo en ~/.config/kitty/kitty.conf
# Paquete debe ser:
kitty/
  .config/                                     # replica la estructura
    kitty/
      kitty.conf

```

```
# NO:
kitty/
    kitty.conf          # falta .config/
```

4.5 3. Agrupar Lógicamente

```
# Opción 1: Por aplicación
~/dotfiles/
  nvim/
  vim/
  emacs/

# Opción 2: Por categoría (menos común)
~/dotfiles/
  editors/
    .vimrc
    .config/nvim/
  shells/
    .zshrc
    .bashrc

# Recomendado: Opción 1 (por aplicación)
```

4.6 Ejemplos de Estructuras

4.7 Estructura Simple

```
~/dotfiles/
  bash/
    .bashrc
  git/
    .gitconfig
  vim/
    .vimrc
```

Instalación:

```
mkdir ~/dotfiles
cd ~/dotfiles

# Crear la estructura para bash
mkdir -p bash

# o mueve, o crea symlink, como prefieras
cp ~/.bashrc bash/.bashrc
# (opcional) cp ~/.bash_profile bash/.bash_profile
```



```
# Para git
mkdir git
cp ~/.gitconfig git/.gitconfig

# Para vim
mkdir vim
cp ~/.vimrc vim/.vimrc
# si tienes ~/.vim/ con plugins, etc → también lo copias/mueves

# Para zsh + oh-my-zsh customizaciones
mkdir -p zsh/.config
cp ~/.zshrc zsh/.zshrc
```

Una vez que tengas (por ejemplo) la carpeta bash/ con .bashrc dentro:

```
cd ~/dotfiles

# Instalar un paquete
stow bash      # → crea symlink ~/.bashrc → ~/dotfiles/bash/.bashrc
stow git
stow vim

# Instalar múltiples paquetes
stow bash git vim nvim kitty zsh
```

Resultado:

```
~/.bashrc -> dotfiles/bash/.bashrc
~/.gitconfig -> dotfiles/git/.gitconfig
~/.vimrc -> dotfiles/vim/.vimrc
```

4.8 Estructura Compleja

```
~/dotfiles/
  shell/
    .bashrc
    .zshrc
    .config/
      bash/
        aliases.bash
      zsh/
        aliases.zsh
  terminal/
    .config/
      kitty/
        kitty.conf
```

```
    themes/
    alacritty/
        alacritty.yml
editor/
    .config/
        nvim/
            init.lua
            lua/
                plugins.lua
```

5 Instalación de Paquetes

5.1 Proceso de Instalación

5.2 1. Tree Folding (Plegado de Árbol)

Stow intenta crear el **mínimo número de symlinks** posible.

Ejemplo 1: Target Vacío

```
# Estado inicial:
~/ (vacío, sin ~/.config/)

# Paquete:
~/dotfiles/nvim/
    .config/
        nvim/
            init.lua

# Comando:
cd ~/dotfiles
stow nvim

# Resultado (tree folding):
~/ .config -> dotfiles/nvim/.config/

# En lugar de:
# ~/.config/nvim/init.lua -> ...
# Stow crea un symlink al directorio completo
```

Ejemplo 2: Target con Archivos Existentes

```
# Estado inicial:
~/ .config/
    kitty/          # ya existe
        kitty.conf

# Paquete:
~/dotfiles/nvim/
    .config/
```

```
    nvim/  
        init.lua  
  
# Comando:  
stow nvim  
  
# Resultado (NO puede hacer tree folding):  
~/.config/          # directorio real  
    kitty/          # ya existía  
        kitty.conf  
nvim -> ../dotfiles/nvim/.config/nvim/
```

5.3 2. Tree Unfolding (Desplegado de Árbol)

Cuando un symlink plegado debe ser “abierto” para acomodar otro paquete.

Escenario:

```
# Estado inicial:  
~/.config -> dotfiles/nvim/.config/  
  
# Instalar otro paquete:  
~/dotfiles/kitty/  
    .config/  
        kitty/  
            kitty.conf  
  
# Comando:  
stow kitty  
  
# Proceso de unfolding:  
# 1. Eliminar symlink: ~/.config  
# 2. Crear directorio: ~/.config/  
# 3. Crear symlinks:  
#     ~/.config/nvim -> ../dotfiles/nvim/.config/nvim/  
#     ~/.config/kitty -> ../dotfiles/kitty/.config/kitty/
```

5.4 Instalación Básica

```
# Crea el directorio  
mkdir ~/dotfiles  
  
# Navegar al stow directory  
cd ~/dotfiles  
  
# Instalar un paquete  
stow nvim
```

```
# Instalar múltiples paquetes
stow nvim zsh git kitty

# Instalar todos los paquetes
stow */
```

5.5 Instalación con Verificación

```
# Dry run primero (simular)
stow -nv nvim

# Si todo OK, instalar realmente
stow nvim

# Verificar symlinks creados
ls -la ~/.config/nvim
```

5.6 Instalación Selectiva

```
# Solo paquetes de terminal
stow kitty alacritty tmux

# Solo paquetes de shell
stow bash zsh fish

# Solo paquetes de editor
stow nvim vim emacs
```

6 Desinstalación de Paquetes

6.1 Proceso de Desinstalación

6.2 1. Eliminación de Symlinks

```
# Paquete instalado:
~/.zshrc -> dotfiles/zsh/.zshrc

# Desinstalar:
cd ~/dotfiles
stow -D zsh

# Resultado:
# ~/.zshrc eliminado (porque era symlink a stow package)
```

6.3 2. Eliminación de Directorios Vacíos

```
# Antes:
~/.config/
  nvim -> ../dotfiles/nvim/.config/nvim/

# Desinstalar:
stow -D nvim

# Después:
# ~/.config/ eliminado (si quedó vacío)
```

6.4 3. Tree Refolding (Re-plegado)

Después de eliminar symlinks, si un directorio contiene solo symlinks a un único paquete, Stow lo “re-plega”.

Escenario:

```
# Estado actual:
~/.config/
  nvim -> ../dotfiles/nvim/.config/nvim/
  kitty -> ../dotfiles/kitty/.config/kitty/

# Desinstalar kitty:
stow -D kitty

# Resultado (refolding):
~/.config -> dotfiles/nvim/.config/
```

6.5 Desinstalación Básica

```
# Navegar al stow directory
cd ~/dotfiles

# Desinstalar un paquete
stow -D nvim

# Desinstalar múltiples paquetes
stow -D nvim zsh git

# Desinstalar todos los paquetes
stow -D */
```

6.6 Desinstalación con Verificación

```
# Dry run primero
stow -Dnv nvim
```

```
# Si todo OK, desinstalar realmente
stow -D nvim

# Verificar que symlinks fueron eliminados
ls -la ~/.config/nvim
```

6.7 Desinstalación Parcial

```
# Desinstalar solo configuraciones de terminal
stow -D kitty alacritty tmux

# Mantener el resto
```

7 Reinstalación de Paquetes

7.1 Comando Restow

```
# Restow = Unstow + Stow
stow -R nvim
stow --restow nvim
```

7.2 Cuándo Usar Restow

1. Después de actualizar un paquete:

```
# Editaste archivos en ~/dotfiles/nvim/
cd ~/dotfiles
stow -R nvim

# Esto actualiza los symlinks si la estructura cambió
```

2. Para limpiar symlinks obsoletos:

```
# Eliminaste archivos del paquete
stow -R nvim

# Restow elimina symlinks huérfanos
```

3. Después de cambiar estructura:

```
# Moviste archivos dentro del paquete
# Antes: nvim/.vimrc
# Ahora: nvim/.config/nvim/init.lua

stow -R nvim
```

7.3 Restow vs Delete + Stow

```
# Método 1: Restow (recomendado)
stow -R nvim

# Método 2: Manual (equivalente)
stow -D nvim
stow nvim

# Ventaja de -R: más rápido, optimizado
```

8 Gestión de Dotfiles

8.1 Setup Inicial

8.2 1. Crear Estructura

```
# Crear directorio para dotfiles
mkdir -p ~/.dotfiles
cd ~/.dotfiles

# Inicializar Git
git init
```

8.3 2. Mover Configuraciones Existentes

Método manual:

```
# Crear paquete
mkdir -p ~/.dotfiles/zsh

# Mover archivos
mv ~/.zshrc ~/.dotfiles/zsh/
mv ~/.zshenv ~/.dotfiles/zsh/

# Stow
cd ~/.dotfiles
stow zsh
```

Con script:

```
#!/bin/bash
# migrate-to-stow.sh

DOTFILES="$HOME/.dotfiles"
mkdir -p "$DOTFILES"

# Migrar zsh
mkdir -p "$DOTFILES/zsh"
mv ~/.zshrc "$DOTFILES/zsh/"
```

```
mv ~/.zshenv "$DOTFILES/zsh/"

# Migrar nvim
mkdir -p "$DOTFILES/nvim/.config"
mv ~/.config/nvim "$DOTFILES/nvim/.config/"

# Migrar git
mkdir -p "$DOTFILES/git"
mv ~/.gitconfig "$DOTFILES/git/"

# Stow todo
cd "$DOTFILES"
stow zsh nvim git
```

8.4 3. Usar --adopt (Con Precaución)

```
# Crear estructura primero
mkdir -p ~/dotfiles/nvim/.config
mkdir ~/dotfiles/nvim/.config/nvim

# Adoptar configuración existente
cd ~/dotfiles
stow --adopt nvim

# Esto MUEVE ~/.config/nvim/* a ~/dotfiles/nvim/.config/nvim/
# Y luego crea el symlink
```

8.5 Workflow Diario

8.6 Editar Configuraciones

```
# Los symlinks te permiten editar en cualquier lugar:

# Opción 1: Editar en home (a través del symlink)
nvim ~/.zshrc # Edita ~/dotfiles/zsh/.zshrc

# Opción 2: Editar directamente en dotfiles
nvim ~/dotfiles/zsh/.zshrc # Mismo archivo
```

8.7 Agregar Nueva Aplicación

```
# 1. Crear paquete
cd ~/dotfiles
mkdir -p new-app/.config/new-app

# 2. Agregar archivos
```



```
cp -r ~/.config/new-app/* new-app/.config/new-app/

# 3. Remover originales
rm -rf ~/.config/new-app

# 4. Stow
stow new-app

# 5. Commit a Git
git add new-app/
git commit -m "Add new-app configuration"
```

8.8 Sincronizar con Git

```
cd ~/dotfiles

# Después de cambios
git add .
git commit -m "Update nvim configuration"
git push origin main

# En otra máquina
git pull origin main
stow nvim # o stow -R nvim si ya estaba instalado
```

8.9 Manejo de Archivos Sensibles

8.10 Estrategia 1: .gitignore

```
# ~/dotfiles/.gitignore
# Ignorar archivos sensibles

# SSH keys
.ssh/id_*
.ssh/*.pem

# Contraseñas
.netrc
.authinfo

# Tokens
.config/gh/hosts.yml
```

8.11 Estrategia 2: Archivos Template

```
# Crear template sin datos sensibles
# ~/dotfiles/git/.gitconfig.local.template
[user]
    name = YOUR_NAME
    email = YOUR_EMAIL

# .gitignore
.gitconfig.local

# Script de setup
#!/bin/bash
if [ ! -f ~/dotfiles/git/.gitconfig.local ]; then
    cp ~/dotfiles/git/.gitconfig.local.template \
        ~/dotfiles/git/.gitconfig.local
    echo "Edit ~/dotfiles/git/.gitconfig.local"
fi
```

8.12 Estrategia 3: Encriptación

```
# Usar git-crypt o similar
cd ~/dotfiles
git-crypt init

# Especificar qué encriptar
# .gitattributes
.netrc filter=git-crypt diff=git-crypt
.ssh/id_* filter=git-crypt diff=git-crypt
```

8.13 Estructura para Múltiples Hosts

```
~/dotfiles/
  common/           # Compartido entre todos
    git/
    tmux/
  desktop/          # Solo desktop
    kde/
    i3/
  laptop/            # Solo laptop
    power-management/
  server/            # Solo servers
    ssh/
```

Script de instalación por host:

```
#!/bin/bash
# install.sh
```

```

HOSTNAME=$(hostname)

# Instalar común
cd ~/dotfiles/common
stow */

# Instalar específico del host
case "$HOSTNAME" in
    desktop-main)
        cd ~/dotfiles/desktop
        stow */
        ;;
    laptop-work)
        cd ~/dotfiles/laptop
        stow */
        ;;
    server-*)
        cd ~/dotfiles/server
        stow */
        ;;
esac

```

9 Ignore Lists

9.1 Tipos de Ignore Lists

9.2 1. Built-in (Predeterminado)

Stow ignora automáticamente:

```

RCS
.+,v
CVS
\.\#.+      # CVS conflict files / emacs lock files
\cvsignore
\.svn
_darcs
\.hg
\.git
\.gitignore
\.gitmodules
.+~        # emacs backup files
\#.*\#     # emacs autosave files
~/README.*
~/LICENSE.*
~/COPYING

```

9.3 2. Global Ignore List

Archivo: ~/.stow-global-ignore

```
# ~/.stow-global-ignore

# Archivos de respaldo
.*\bak
.*\old
.*\orig

# Temporales
.*\swp
.*\tmp

# OS específicos
\DS_Store
Thumbs\db

# IDEs
\idea
\vscode

# Build artifacts
node_modules
__pycache__
*.pyc
```

9.4 3. Package-Local Ignore List

Archivo: <package>/.stow-local-ignore

```
# ~/dotfiles/nvim/.stow-local-ignore

# Plugin managers
^/\config/nvim/plugin/packer_compiled.lua

# Cache
^/\config/nvim/.*\cache/

# Logs
^/\config/nvim/.*\log

# Lazy-lock
^/\config/nvim/lazy-lock.json
```

9.5 Sintaxis de Ignore Lists

9.6 Reglas de Matching

1. Expresiones con / (path completo):

```
# Match contra path completo desde raíz del paquete
~/README.*          # README en raíz
~/\.\config/nvim/cache/ # Directorio cache específico
```

2. Expresiones sin / (basename):

```
# Match contra nombre del archivo/directorio
README.*          # Cualquier README en cualquier ubicación
.*\.\log          # Archivos .log en cualquier ubicación
```

9.7 Ejemplos Prácticos

Ejemplo 1: Ignorar documentación:

```
# .stow-local-ignore
~/README.*
~/LICENSE.*
~/CHANGELOG.*
~/docs/
```

Ejemplo 2: Ignorar archivos temporales:

```
# .stow-local-ignore
.*\.\swp$
.*\.\swo$
.*~$
\#.*\#$
```

Ejemplo 3: Ignorar por aplicación:

```
# nvim/.stow-local-ignore
~/\.\config/nvim/plugin/
~/\.\config/nvim/.*\.\cache/
lazy-lock\.json

# zsh/.stow-local-ignore
\.\zcompdump
\.\zsh_history
```

9.8 Precedencia de Ignore Lists

1. .stow-local-ignore (en paquete)
 - ↓ (si no existe)
2. ~/.stow-global-ignore
 - ↓ (si no existe)
3. Built-in ignore list

9.9 Opción --ignore en CLI

```
# Ignorar específicos para esta ejecución
stow --ignore='.*\.orig' --ignore='.*\.dist' nvim

# Equivalente a expresión OR
stow --ignore='.*\.orig|.*\.dist' nvim

# Combina con ignore lists existentes
```

10 Opciones Avanzadas

10.1 Tree Folding Control

10.2 --no-folding

Desactiva tree folding completamente.

Sin --no-folding (default):

```
# Resultado:
~/.config -> dotfiles/nvim/.config/
```

Con --no-folding:

```
# Resultado:
~/.config/                # directorio real
  nvim -> ../dotfiles/nvim/.config/nvim/
```

Uso:

```
stow --no-folding nvim
```

10.3 Adopt Mode

10.4 --adopt

ADVERTENCIA: Modifica el contenido del stow directory.

Escenario:

```
# Tienes configuración existente:
~/.config/nvim/init.lua

# Quieres adoptarla en tu paquete:
~/dotfiles/nvim/.config/nvim/ (vacío)

# Comando:
cd ~/dotfiles
stow --adopt nvim

# Resultado:
# 1. ~/.config/nvim/init.lua → movido a ~/dotfiles/nvim/.config/nvim/init.lua
# 2. ~/.config/nvim/init.lua → se convierte en symlink
```

Uso con Git:

```
# 1. Adoptar archivos
stow --adopt nvim

# 2. Ver diferencias
cd nvim
git diff

# 3. Decidir qué mantener
git add -p # Añadir selectivamente
# o
git checkout HEAD -- . # Descartar cambios adoptados
```

10.5 Defer y Override**10.6 --defer**

Evita stowing si el archivo ya está stowed por otro paquete.

Escenario:

```
# paquete-a tiene:
paquete-a/
  .config/
    shared/
      config.txt

# paquete-b tiene:
paquete-b/
  .config/
    shared/
      config.txt

# Instalar A primero:
stow paquete-a # OK

# Instalar B con defer:
stow --defer='.config/shared/config.txt' paquete-b
# B no sobrescribirá config.txt de A
```

10.7 --override

Fuerza stowing incluso si ya existe symlink de otro paquete.

Escenario:

```
# Mismo escenario de arriba

# Instalar B con override:
stow --override='.config/shared/' paquete-b
# B sobrescribirá todos los archivos en .config/shared/
```

10.8 Dotfiles Mode

10.9 --dotfiles

Transforma dot- en . al hacer stow.

Uso:

```
# Estructura del paquete:
bash/
  dot-bashrc
  dot-bash_profile
  dot-config/
    bash/
      aliases.bash

# Stow con --dotfiles:
stow --dotfiles bash

# Resultado:
~/.bashrc -> dotfiles/bash/dot-bashrc
~/.bash_profile -> dotfiles/bash/dot-bash_profile
~/.config/ ...
```

Ventajas:

- Mantiene paquetes visibles (no ocultos por .)
- Más fácil navegar en GUI
- Mejor para Git

Desventajas:

- Necesita usar --dotfiles siempre
- Puede confundir
- No estándar

Recomendación: Usar nombres normales con . en vez de dot-.

10.10 Multiple Stow Directories

Puedes tener múltiples stow directories para diferentes propósitos.

Ejemplo:

```
# Estructura:
~/dotfiles/           # Personal configs
  nvim/

~/work-dotfiles/      # Work configs
  nvim/

# Marcar como stow directories:
```



```
touch ~/dotfiles/.stow
touch ~/work-dotfiles/.stow

# Stow desde diferentes directorios:
cd ~/dotfiles && stow nvim
cd ~/work-dotfiles && stow nvim
```

.stow file: Indica que un directorio es stow directory, protegiéndolo de operaciones de unstow.

11 Casos de Uso Prácticos

11.1 Caso 1: Setup en Nueva Máquina

Objetivo: Instalar dotfiles en computadora nueva.

```
# 1. Clonar repositorio
cd ~
git clone https://github.com/user/dotfiles.git

# 2. Instalar dependencias (si hay)
cd dotfiles
./scripts/install-deps.sh # Opcional

# 3. Hacer backup de existentes (precaución)
mkdir -p ~/backup-dotfiles
cp ~/.zshrc ~/backup-dotfiles/ # etc

# 4. Stow paquetes deseados
stow zsh nvim git kitty tmux

# 5. Verificar
ls -la ~/.zshrc
# ~/.zshrc -> dotfiles/zsh/.zshrc

# 6. Restart shell o source
source ~/.zshrc
```

Script automatizado:

```
#!/bin/bash
# install.sh

set -e

DOTFILES="$HOME/dotfiles"
BACKUP="$HOME/dotfiles-backup-$(date +%Y%m%d)"

# Backup existentes
```

```
if [ -f ~/.zshrc ]; then
    mkdir -p "$BACKUP"
    cp ~/.zshrc "$BACKUP/"
    echo "Backed up existing .zshrc"
fi

# Stow
cd "$DOTFILES"
stow zsh nvim git tmux kitty

echo "Dotfiles installed!"
echo "Backup location: $BACKUP"
```

11.2 Caso 2: Actualizar Configuraciones

Objetivo: Actualizar configs en máquina existente.

```
# 1. Pull latest changes
cd ~/dotfiles
git pull origin main

# 2. Restow para actualizar symlinks
stow -R nvim zsh

# 3. Si estructura cambió mucho:
stow -D nvim # Remover viejo
stow nvim    # Instalar nuevo

# 4. Verificar cambios
nvim ~/.config/nvim/init.lua
```

11.3 Caso 3: Probar Nueva Configuración

Objetivo: Probar configuración sin afectar la actual.

```
# 1. Crear branch en Git
cd ~/dotfiles
git checkout -b test-new-config

# 2. Modificar paquete
nvim nvim/.config/nvim/init.lua

# 3. Restow
stow -R nvim

# 4. Probar
nvim
```

```
# 5a. Si funciona bien:
git checkout main
git merge test-new-config
git push

# 5b. Si no funciona:
git checkout main
stow -R nvim # Vuelve a config anterior
```

11.4 Caso 4: Compartir Configuración entre Usuarios

Objetivo: Múltiples usuarios con dotfiles compartidos.

Setup:

```
# Ubicación compartida
sudo mkdir -p /opt/shared-dotfiles
sudo chown :developers /opt/shared-dotfiles
sudo chmod g+w /opt/shared-dotfiles

# User A: Setup inicial
cd /opt/shared-dotfiles
git init
mkdir -p common/git common/tmux
# ... agregar configs ...
git add .
git commit -m "Initial shared configs"

# User B: Usar configs
cd /opt/shared-dotfiles
git pull
stow -d /opt/shared-dotfiles -t ~ common/git common/tmux
```

11.5 Caso 5: Configuración Por Entorno

Objetivo: Diferentes configs para diferentes entornos.

Estructura:

```
~/dotfiles/
  common/      # Común a todos
    git/
    tmux/
  dev/         # Desarrollo
    nvim/
  prod/        # Producción
    nvim/
  install-env.sh
```

Script:

```
#!/bin/bash
# install-env.sh

ENV="${1:-dev}" # default: dev

cd ~/dotfiles

# Instalar común
stow -d common -t ~ */

# Instalar específico del entorno
stow -d "$ENV" -t ~ */

echo "Environment: $ENV installed"
```

Uso:

```
# Desarrollo
./install-env.sh dev

# Producción
./install-env.sh prod
```

12 Tu Repositorio .dotfiles

Voy a analizar tu repositorio y dar recomendaciones específicas.

12.1 Análisis de tu Estructura Actual

```
~/dotfiles/
git/
  .gitconfig
kde/
  .config/
    kdeglobals
    dolphinrc
    ...
shell/
  .zshrc
  starship.toml
terminal/
  .config/
    konsolerc
vscode/
  .config/
    settings.json
    keybindings.json
```

```

zotero/
  .zotero/...
obsidian/
  Documents/thoughts/.obsidian/
... (más paquetes)

```

12.2 Implementación de Stow en tu Repo

12.3 1. Script install.sh Mejorado

Tu `install.sh` actual debe usar Stow. Aquí está mi versión mejorada:

```

#!/bin/bash
# ~/dotfiles/install.sh

set -e

DOTFILES="$HOME/dotfiles"
BACKUP_DIR="$HOME/dotfiles-backup-$(date +%Y%m%d-%H%M%S)"

# Colores
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color

# Funciones
log_info() {
  echo -e "${GREEN}[INFO]${NC} $1"
}

log_warn() {
  echo -e "${YELLOW}[WARN]${NC} $1"
}

log_error() {
  echo -e "${RED}[ERROR]${NC} $1"
}

# Verificar que Stow está instalado
if ! command -v stow &> /dev/null; then
  log_error "Stow no está instalado"
  log_info "Instalando stow..."
  sudo apt update && sudo apt install -y stow
fi

# Función para hacer backup
backup_if_exists() {

```

```

    local file="$1"
    if [ -e "$file" ] && [ ! -L "$file" ]; then
        mkdir -p "$BACKUP_DIR"
        cp -r "$file" "$BACKUP_DIR/"
        log_warn "Backup: $file -> $BACKUP_DIR/"
    fi
}

# Función para stow paquete
stow_package() {
    local package="$1"

    log_info "Stowing $package..."

    # Dry run primero
    if stow -nv "$package" 2>&1 | grep -q "WARNING"; then
        log_warn "Conflicto detectado para $package"
        read -p "¿Hacer backup y continuar? [y/N] " -n 1 -r
        echo
        if [[ $REPLY =~ ^[Yy]$ ]]; then
            # Hacer backup de archivos conflictivos
            # (aquí necesitarías lógica más sofisticada)
            stow "$package"
        else
            log_error "Saltando $package"
            return 1
        fi
    else
        stow "$package"
        log_info " $package instalado"
    fi
}

# Cambiar a dotfiles directory
cd "$DOTFILES" || exit 1

# Lista de paquetes a instalar
PACKAGES=(
    "git"
    "shell"
    "terminal"
    "kde"
    "vscode"
    "nvim"
    "kitty"
    # ... más paquetes
)

```

```
# Opción para instalar todo o selectivo
if [ "$1" == "all" ]; then
    PACKAGES=$(ls -d */ | sed 's#/#')
    log_info "Instalando TODOS los paquetes"
elif [ $# -gt 0 ]; then
    PACKAGES=("$@")
    log_info "Instalando paquetes especificados: ${PACKAGES[*]}"
fi

# Instalar paquetes
for package in "${PACKAGES[@]}; do
    stow_package "$package" || true
done

log_info "Instalación completa!"
if [ -d "$BACKUP_DIR" ]; then
    log_info "Backups guardados en: $BACKUP_DIR"
fi
```

Uso:

```
# Instalar paquetes específicos
./install.sh git shell terminal

# Instalar todo
./install.sh all

# Ver qué haría sin hacer cambios
# (modificar script para agregar -n flag)
```

12.4 2. Script para Desinstalar

```
#!/bin/bash
# ~/dotfiles/uninstall.sh

DOTFILES="$HOME/dotfiles"

cd "$DOTFILES" || exit 1

if [ $# -eq 0 ]; then
    echo "Uso: $0 <paquete1> [paquete2] ..."
    echo "0: $0 all"
    exit 1
fi

if [ "$1" == "all" ]; then
    PACKAGES=$(ls -d */ | sed 's#/#')
```

```

else
    PACKAGES=("$@")
fi

for package in "${PACKAGES[@]}; do
    echo "Unstowing $package..."
    stow -D "$package"
    echo " $package desinstalado"
done

```

12.5 3. Reorganizar Paquetes Problemáticos

Zotero: Ubicación no estándar

```

# Actual:
zotero/
    .zotero/zotero/25vfdnq5.default/
        prefs.js

# Problema: .zotero está en HOME pero tiene subdirectorios profundos

# Solución 1: Usar como está (funciona)
stow zotero
# Resultado: ~/.zotero/... → dotfiles/zotero/.zotero/...

# Solución 2: Si solo quieres prefs.js, simplificar:
zotero/
    .zotero/
        zotero/
            25vfdnq5.default/
                prefs.js

```

Obsidian: Ruta específica

```

# Actual:
obsidian/
    Documents/thoughts/.obsidian/

# Problema: No está en .config sino en Documents

# Solución: Está bien así, Stow lo maneja
stow obsidian
# Resultado: ~/Documents/thoughts/.obsidian → ...

```

12.6 4. .stowrc para tu Repo

Crear ~/dotfiles/.stowrc:


```
# ~/dotfiles/.stowrc

# Target es siempre HOME
--target=$HOME

# Ignorar archivos comunes
--ignore='.git'
--ignore='README.*'
--ignore='LICENSE.*'
--ignore='*.swp'
--ignore='.*~'
--ignore='install.sh'
--ignore='uninstall.sh'
--ignore='.stowrc'
```

Con esto, no necesitas especificar `-t ~` cada vez.

12.7 5. .stow-local-ignore por Paquete

Para vscode:

```
# ~/dotfiles/vscode/.stow-local-ignore

# No stow extensiones (solo configuración)
^/\.config/Code/CachedData/
^/\.config/Code/logs/
^/\.config/Code/User/workspaceStorage/
```

Para kde:

```
# ~/dotfiles/kde/.stow-local-ignore

# Archivos de sesión y cache
^/\.config/session/
^/\.cache/
```

Para shell:

```
# ~/dotfiles/shell/.stow-local-ignore

# Historia de shells (puede tener info sensible)
^/\.zsh_history
^/\.bash_history

# Archivos compilados
\.zcompdump
```

12.8 6. Script de Verificación

```
#!/bin/bash
# ~/dotfiles/check-stow.sh

# Verificar qué está stowed

DOTFILES="$HOME/dotfiles"

echo "Paquetes stowed:"
echo "====="

cd "$DOTFILES" || exit 1

for package in */; do
    package=${package%/}

    # Encontrar primer archivo del paquete
    first_file=$(find "$package" -type f | head -1)

    if [ -z "$first_file" ]; then
        continue
    fi

    # Convertir a path en HOME
    home_path="$HOME/${first_file#$package}/"

    if [ -L "$home_path" ]; then
        target=$(readlink "$home_path")
        if [[ "$target" == *"$DOTFILES/$package"* ]]; then
            echo " $package"
        else
            echo " $package (symlink apunta a otro lugar)"
        fi
    else
        echo " $package (no stowed)"
    fi
done
```

12.9 7. Actualizar .gitignore

```
# ~/dotfiles/.gitignore

# Backups
*~
*.bak
*.old
```

```
*.orig
.*.swp

# Datos sensibles
shell/.zsh_history
shell/.bash_history
.netrc
.authinfo

# Cache y temporales
**/.cache/
**/__pycache__/
**/node_modules/

# Logs
**/*.log

# Sistema
.DS_Store
Thumbs.db

# Archivos de Stow
.stow

# Zotero database (demasiado grande)
zotero/.zotero/zotero/*/zotero.sqlite*

# VSCode workspace storage
vscode/.config/Code/User/workspaceStorage/
```

12.10 8. Comandos Útiles para tu Repo

```
# Navegar a dotfiles
cd ~/dotfiles

# Instalar todo (primera vez)
./install.sh all

# Instalar paquetes esenciales
./install.sh git shell terminal kde

# Verificar qué está instalado
./check-stow.sh

# Actualizar después de pull
git pull
stow -R */ # Restow todo
```

```
# Desinstalar temporalmente para pruebas
stow -D vscode
# hacer pruebas...
stow vscode # Reinstalar

# Agregar nuevo paquete
mkdir new-app
# crear estructura...
stow new-app
git add new-app/
git commit -m "Add new-app"
```

13 Workflows Completos

13.1 Workflow 1: Configuración Inicial

```
# Paso 1: Crear estructura
mkdir -p ~/dotfiles
cd ~/dotfiles
git init

# Paso 2: Crear paquetes
mkdir -p zsh nvim git

# Paso 3: Mover configs existentes
mv ~/.zshrc zsh/
mv ~/.config/nvim nvim/.config/
mv ~/.gitconfig git/

# Paso 4: Stow
stow zsh nvim git

# Paso 5: Verificar
ls -la ~/.zshrc # debe ser symlink

# Paso 6: Git
git add .
git commit -m "Initial dotfiles"
git remote add origin git@github.com:user/dotfiles.git
git push -u origin main
```

13.2 Workflow 2: Día a Día

```
# Editar configuración (desde cualquier lugar)
nvim ~/.config/nvim/init.lua # Edita a través del symlink
```

```
# Commit cambios
cd ~/dotfiles
git add nvim/
git commit -m "Update nvim config: add new plugin"
git push

# En otra máquina
cd ~/dotfiles
git pull
# Los cambios se reflejan automáticamente (symlinks)
```

13.3 Workflow 3: Nueva Máquina

```
# Clonar
git clone https://github.com/user/dotfiles.git ~/dotfiles

# Instalar Stow
sudo apt install stow

# Backup existentes (precaución)
mkdir ~/backup
cp ~/.zshrc ~/backup/ 2>/dev/null || true

# Stow
cd ~/dotfiles
stow */

# Verificar
ls -la ~/ | grep '\->'

# Instalar dependencias de apps
# (nvim plugins, zsh plugins, etc)
```

13.4 Workflow 4: Experimentar

```
# Crear branch de experimento
cd ~/dotfiles
git checkout -b experiment-new-nvim

# Modificar libremente
nvim nvim/.config/nvim/init.lua

# Restow para aplicar
stow -R nvim

# Probar...
```

```
# Si funciona:
git checkout main
git merge experiment-new-nvim

# Si no funciona:
git checkout main
stow -R nvim # Vuelve a main automáticamente
```

13.5 Workflow 5: Actualización Limpia

```
# Pull cambios
cd ~/dotfiles
git pull origin main

# Verificar qué cambió
git log -p --since="1 week ago"

# Desinstalar y reinstalar (limpia symlinks obsoletos)
stow -D nvim
stow nvim

# O usar restow
stow -R nvim

# Verificar que funciona
nvim --version
```

14 Integración con Git

14.1 Estructura de Repositorio

```
~/dotfiles/
.git/
.gitignore
.stowrc
README.md
LICENSE
install.sh
uninstall.sh
check-stow.sh
zsh/
  .stow-local-ignore
  .zshrc
  .zshenv
nvim/
  .stow-local-ignore
```

```
.config/  
  nvim/  
... (más paquetes)
```

14.2 .gitignore Completo

```
# ~/dotfiles/.gitignore  
  
# =====  
# BACKUPS  
# =====  
*~  
*.bak  
*.old  
*.orig  
*.swp  
*.swo  
  
# =====  
# HISTORIA Y DATOS SENSIBLES  
# =====  
  
# Shell history (puede contener comandos con passwords)  
**/.zsh_history  
**/.bash_history  
**/.history  
  
# Credenciales  
.netrc  
.authinfo  
**/.ssh/id_*  
**/.ssh/*.pem  
  
# Tokens  
**/.config/gh/hosts.yml  
  
# =====  
# CACHE Y TEMPORALES  
# =====  
  
# Directorios de cache  
**/.cache/  
**/__pycache__/  
**/node_modules/  
  
# Compilados  
*.pyc
```

```

*.zwc
.zcompdump*

# Logs
**/*.log

# =====
# ARCHIVOS DE SISTEMA
# =====
.DS_Store
Thumbs.db
desktop.ini

# =====
# STOW
# =====
.stow

# =====
# APLICACIONES ESPECÍFICAS
# =====

# Zotero (database muy grande)
zotero/.zotero/zotero/*/zotero.sqlite*
zotero/.zotero/zotero/*/storage/

# VSCode
vscode/.config/Code/User/workspaceStorage/
vscode/.config/Code/CachedData/
vscode/.config/Code/logs/

# Obsidian
obsidian/Documents/thoughts/.obsidian/workspace
obsidian/Documents/thoughts/.obsidian/workspace.json

# KDE
kde/.config/session/
kde/.cache/

```

14.3 Commits Best Practices

```

# Commits semánticos

# Agregar nueva aplicación
git commit -m "feat(tmux): Add tmux configuration"

# Actualizar configuración

```



```
git commit -m "chore(nvim): Update LSP settings"

# Fix
git commit -m "fix(zsh): Correct path to starship"

# Documentación
git commit -m "docs: Update README with stow instructions"

# Refactor
git commit -m "refactor(shell): Reorganize shell configs"
```

14.4 Branches Strategy

```
# Main branch
main # Configuración estable

# Feature branches
feature/add-tmux-config
feature/new-nvim-setup

# Experimental
experiment/test-fish-shell
experiment/new-colorscheme

# Host-specific
host/desktop-main
host/laptop-work
host/server-prod
```

14.5 Tags para Versiones

```
# Taggear versiones estables
git tag -a v1.0.0 -m "Stable dotfiles v1.0.0"
git push origin v1.0.0

# Ver tags
git tag -l

# Checkout a tag
git checkout v1.0.0
```

14.6 Submodules para Plugins

```
# Agregar plugin como submodule
cd ~/dotfiles/nvim/.config/nvim
```

```
git submodule add https://github.com/user/plugin.git pack/plugins/start/plugin

# Actualizar submodules
git submodule update --init --recursive

# Pull con submodules
git pull --recurse-submodules
```

14.7 GitHub Actions para Validación

```
# .github/workflows/validate.yml

name: Validate Dotfiles

on: [push, pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v2

      - name: Install stow
        run: sudo apt-get install -y stow

      - name: Test stow (dry run)
        run: |
          cd $GITHUB_WORKSPACE
          stow -nv */

      - name: Check for sensitive data
        run: |
          # Verificar que no haya claves SSH
          if find . -name "id_rsa" -o -name "id_ed25519"; then
            echo "ERROR: SSH keys found!"
            exit 1
          fi
```

15 Troubleshooting

15.1 Problema 1: Conflictos al Stow

Error:

```
WARNING! stowing nvim would cause conflicts:
* existing target is neither a link nor a directory: .config/nvim/init.lua
All operations aborted.
```

Causa: Ya existe un archivo/directorio en el target que no es un symlink de Stow.

Soluciones:

Opción 1: Hacer backup y eliminar

```
# Backup
cp ~/.config/nvim/init.lua ~/.config/nvim/init.lua.backup

# Eliminar
rm ~/.config/nvim/init.lua

# Stow
stow nvim
```

Opción 2: Usar `--adopt` (cuidado)

```
stow --adopt nvim
# Mueve el archivo al paquete y crea symlink
```

Opción 3: Verificar y resolver manualmente

```
# Ver qué está causando conflicto
stow -nv nvim

# Resolver caso por caso
```

15.2 Problema 2: Symlinks Rotos

Error:

```
ls -la ~/.zshrc
# lrwxrwxrwx ... .zshrc -> dotfiles/zsh/.zshrc (broken)
```

Causa: El paquete fue movido o eliminado.

Soluciones:

Opción 1: Restow

```
cd ~/dotfiles
stow -R zsh
```

Opción 2: Desinstalar y reinstalar

```
stow -D zsh # Limpia symlinks rotos
stow zsh    # Crea nuevos
```

Opción 3: Encontrar todos los symlinks rotos

```
# Encontrar symlinks rotos en HOME
find ~/ -xtype l

# Eliminar symlinks rotos de Stow
find ~/ -xtype l -lname '*dotfiles*' -delete
```

15.3 Problema 3: Directorio No Vacío

Error:

BUG in find_stowed_path? Absolute/relative mismatch

Causa: Stow está confundido por la estructura de directorios.

Solución:

```
# Verificar que estás en el stow directory
pwd # Debe ser ~/dotfiles

# Verificar estructura del paquete
tree nvim

# Usar paths correctos
cd ~/dotfiles
stow -t ~ nvim
```

15.4 Problema 4: Tree Folding Inesperado

Problema: Stow crea symlink a directorio completo en lugar de entrar y enlazar archivos.

Ejemplo:

```
# Esperado:
~/.config/
  nvim/ (directorio)
    init.lua -> ~/dotfiles/nvim/.config/nvim/init.lua

# Obtenido:
~/.config/
  nvim -> ~/dotfiles/nvim/.config/nvim/ (symlink a directorio)
```

Causa: Stow hace tree folding por defecto para minimizar symlinks.

Solución si no lo quieres:

```
# Usar --no-folding
stow --no-folding nvim

# O desplegar manualmente
stow -D nvim # Remover
mkdir -p ~/.config/nvim # Crear directorio
stow --no-folding nvim # Stow sin folding
```

15.5 Problema 5: Permiso Denegado

Error:

cannot stow: permission denied

Causa: No tienes permisos para crear symlinks en target directory.

Soluciones:

Para /usr/local:

```
# Cambiar ownership
sudo chown -R $USER:$USER /usr/local

# O usar sudo (no recomendado)
sudo stow -t /usr/local myapp
```

Para HOME:

```
# Verificar ownership
ls -ld ~
# drwxr-xr-x 50 user user ...

# Si no eres owner:
sudo chown -R $USER:$USER ~
```

15.6 Problema 6: Stow No Encuentra Paquete

Error:

```
stow: Cannot read package description: No such file or directory
```

Causa: No estás en el stow directory o el paquete no existe.

Solución:

```
# Verificar ubicación
pwd

# Listar paquetes disponibles
ls -ld */

# Cambiar a stow directory
cd ~/dotfiles

# Stow
stow nvim
```

15.7 Problema 7: .stowrc No Se Aplica

Problema: Las opciones en .stowrc no se usan.

Causas y soluciones:

1. Archivo en ubicación incorrecta:

```
# .stowrc debe estar en:  
# - Directorio actual (donde ejecutas stow)  
# - 0 ~/
```



```
# Verificar:  
ls -la .stowrc  
ls -la ~/.stowrc
```

2. Sintaxis incorrecta:

```
# Correcto:  
--target=/home/user
```



```
# Incorrecto:  
target=/home/user # Sin --
```

3. Variables no expandidas:

```
# Use $HOME con comillas si es necesario  
--target=$HOME
```

15.8 Problema 8: Stow Muy Lento

Causa: Directorios muy grandes o muchos archivos.

Soluciones:

1. Usar ignore lists:

```
# Ignorar directorios grandes  
# ~/.stow-global-ignore  
node_modules  
__pycache__  
.cache  
storage
```

2. Evitar stowing todo junto:

```
# En lugar de:  
stow */ # Lento si hay muchos paquetes
```



```
# Hacer:  
stow nvim zsh git # Solo los necesarios
```

3. Simplificar estructura:

```
# Dividir paquetes grandes en paquetes más pequeños
```

16 Scripts de Automatización

16.1 Script 1: install.sh Completo

Ya proporcioné un ejemplo arriba. Aquí una versión más robusta:

```
#!/bin/bash
# ~/dotfiles/install.sh

set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
DOTFILES="$SCRIPT_DIR"
BACKUP_DIR="$HOME/dotfiles-backup-$(date +%Y%m%d-%H%M%S)"
LOG_FILE="$DOTFILES/install.log"

# Colores
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

# Logging
log() {
    echo -e "$1" | tee -a "$LOG_FILE"
}

log_info() {
    log "${BLUE}[$(date +%Y-%m-%d %H:%M:%S)]${NC} ${GREEN}[INFO]${NC} $1"
}

log_warn() {
    log "${BLUE}[$(date +%Y-%m-%d %H:%M:%S)]${NC} ${YELLOW}[WARN]${NC} $1"
}

log_error() {
    log "${BLUE}[$(date +%Y-%m-%d %H:%M:%S)]${NC} ${RED}[ERROR]${NC} $1"
}

# Verificar dependencias
check_dependencies() {
    log_info "Verificando dependencias..."

    if ! command -v stow &> /dev/null; then
        log_error "Stow no está instalado"
        read -p "¿Instalar stow? [Y/n] " -n 1 -r
        echo
        if [[ ! $REPLY =~ ^[Nn]$ ]]; then

```

```

        if command -v apt &> /dev/null; then
            sudo apt update && sudo apt install -y stow
        elif command -v pacman &> /dev/null; then
            sudo pacman -S stow
        elif command -v brew &> /dev/null; then
            brew install stow
        else
            log_error "No se pudo instalar stow automáticamente"
            exit 1
        fi
    else
        exit 1
    fi
fi

log_info " Dependencias OK"
}

# Backup de archivo/directorio existente
backup_if_exists() {
    local path="$1"
    local name="$2"

    if [ -e "$path" ] && [ ! -L "$path" ]; then
        log_warn "Existe: $path"
        mkdir -p "$BACKUP_DIR"
        cp -r "$path" "$BACKUP_DIR/"
        log_info "Backup: $name → $BACKUP_DIR/"
        return 0
    fi
    return 1
}

# Verificar conflictos antes de stow
check_conflicts() {
    local package="$1"

    if stow -nv "$package" 2>&1 | grep -q "WARNING\|ERROR"; then
        return 1
    fi
    return 0
}

# Stow paquete con manejo de errores
stow_package() {
    local package="$1"
    local force="${2:-false}"

```



```

if [ ! -d "$package" ]; then
    log_error "Paquete no existe: $package"
    return 1
fi

log_info "Procesando: $package"

# Check conflicts
if ! check_conflicts "$package"; then
    log_warn "Conflictos detectados en: $package"

    if [ "$force" = "true" ]; then
        log_info "Forzando instalación..."
        # Aquí podrías implementar lógica de backup automático
    else
        read -p "¿Continuar? [y/N] " -n 1 -r
        echo
        if [[ ! $REPLY =~ ^[Yy]$ ]]; then
            log_error "Saltado: $package"
            return 1
        fi
    fi
fi

# Stow
if stow -v "$package"; then
    log_info " Instalado: $package"
    return 0
else
    log_error " Error al instalar: $package"
    return 1
fi
}

```

Mostrar ayuda

```
show_help() {
    cat << EOF
```

Uso: \$0 [opciones] [paquetes...]

Opciones:

-h, --help	Mostrar esta ayuda
-a, --all	Instalar todos los paquetes
-f, --force	Forzar instalación (saltar prompts)
-l, --list	Listar paquetes disponibles
-d, --dry-run	Simular sin hacer cambios

Ejemplos:

```

$0 nvim zsh git      # Instalar paquetes específicos
$0 --all             # Instalar todo
$0 --list            # Ver paquetes disponibles
EOF
}

# Listar paquetes disponibles
list_packages() {
    log_info "Paquetes disponibles:"
    cd "$DOTFILES"
    for package in */; do
        package=${package%/}
        if [ "$package" != ".git" ] && [ -d "$package" ]; then
            echo "  - $package"
        fi
    done
}

# Main
main() {
    local force=false
    local dry_run=false
    local packages=()

    # Parse argumentos
    while [[ $# -gt 0 ]]; do
        case $1 in
            -h|--help)
                show_help
                exit 0
                ;;
            -a|--all)
                cd "$DOTFILES"
                packages=( $(ls -d */ | sed 's#/#' | grep -v '^\.') )
                shift
                ;;
            -f|--force)
                force=true
                shift
                ;;
            -l|--list)
                list_packages
                exit 0
                ;;
            -d|--dry-run)
                dry_run=true
                shift

```

```

        ;;
    *)
        packages+=("$1")
        shift
        ;;
    esac
done

# Verificar que hay paquetes para instalar
if [ ${#packages[@]} -eq 0 ]; then
    log_error "No se especificaron paquetes"
    show_help
    exit 1
fi

# Iniciar log
log_info "=== Instalación de Dotfiles ==="
log_info "Directorio: $DOTFILES"
log_info "Paquetes: ${packages[*]}"

# Verificar dependencias
check_dependencies

# Cambiar a dotfiles directory
cd "$DOTFILES" || exit 1

# Dry run si se especificó
if [ "$dry_run" = true ]; then
    log_info "=== DRY RUN ==="
    for package in "${packages[@]"; do
        log_info "Simulating: $package"
        stow -nv "$package" || true
    done
    exit 0
fi

# Instalar paquetes
local success=0
local failed=0

for package in "${packages[@]"; do
    if stow_package "$package" "$force"; then
        ((success++))
    else
        ((failed++))
    fi
done

```

```
# Resumen
log_info ""
log_info "=== Resumen ==="
log_info "Exitosos: $success"
if [ $failed -gt 0 ]; then
    log_warn "Fallidos: $failed"
fi

if [ -d "$BACKUP_DIR" ]; then
    log_info "Backups en: $BACKUP_DIR"
fi

log_info "Log completo en: $LOG_FILE"
}

# Ejecutar
main "$@"
```

16.2 Script 2: update.sh

```
#!/bin/bash
# ~/dotfiles/update.sh

set -e

DOTFILES="$HOME/dotfiles"

cd "$DOTFILES"

echo " Actualizando dotfiles..."

# Pull latest changes
git pull origin main

# Restow todos los paquetes instalados
for package in */; do
    package=${package%/}

    # Verificar si está stowed
    if find "$HOME" -maxdepth 2 -type l -lname "$DOTFILES/$package/*" 2>/dev/null |
        echo " Restowing $package..."
        stow -R "$package"
    fi
done

echo " Actualización completa!"
```

16.3 Script 3: check.sh

```
#!/bin/bash
# ~/dotfiles/check.sh

DOTFILES="$HOME/dotfiles"

echo " Estado de paquetes:"
echo "===== "

cd "$DOTFILES"

for package in */; do
    package=${package%/}

    if [ "$package" = ".git" ]; then
        continue
    fi

    # Buscar primer archivo del paquete
    first_file=$(find "$package" -type f -o -type l | head -1)

    if [ -z "$first_file" ]; then
        echo " $package (vacío)"
        continue
    fi

    # Convertir a path en HOME
    home_path="$HOME/${first_file#$package}/"

    if [ -L "$home_path" ]; then
        target=$(readlink "$home_path")
        if [[ "$target" == "$DOTFILES/$package" ]]; then
            echo " $package"
        else
            echo " $package (symlink apunta a: $target)"
        fi
    elif [ -e "$home_path" ]; then
        echo " $package (existe pero no es symlink)"
    else
        echo " $package (no instalado)"
    fi
done

# Verificar symlinks rotos
echo ""
echo " Verificando symlinks rotos..."
broken_links=$(find "$HOME" -maxdepth 3 -xtype l -lname "$DOTFILES/*" 2>/dev/null)
```

```
if [ -z "$broken_links" ]; then
    echo "  No hay symlinks rotos"
else
    echo "  Symlinks rotos encontrados:"
    echo "$broken_links"
fi
```

16.4 Script 4: clean.sh

```
#!/bin/bash
# ~/dotfiles/clean.sh

DOTFILES="$HOME/dotfiles"

echo "  Limpiando symlinks huérfanos..."

# Encontrar symlinks rotos que apuntan a dotfiles
find "$HOME" -maxdepth 3 -xtype l -lname "$DOTFILES/*" 2>/dev/null | while read -r b
    echo "Eliminando: $broken_link"
    rm "$broken_link"
done

echo "  Limpieza completa!"
```

17 Best Practices

17.1 1. Organización de Paquetes

DO:

```
# Un paquete por aplicación
~/dotfiles/
  nvim/
  zsh/
  git/

# Replicar estructura de HOME exactamente
nvim/
  .config/
    nvim/
      init.lua
```

DON'T:

```
# Múltiples aplicaciones en un paquete
~/dotfiles/
  configs/
```

```

.config/nvim/
.config/kitty/
.zshrc

# Estructura diferente a HOME
nvim/
  init.lua # Falta .config/nvim/

```

17.2 2. Uso de Ignore Lists

DO:

```

# Ignorar archivos sensibles
# .stow-global-ignore
**/.history
**/.ssh/id_*
.netrc

# Ignorar cache por paquete
# nvim/.stow-local-ignore
~/\ .config/nvim/plugin/packer_compiled\ .lua

```

DON'T:

```

# Commit archivos sensibles sin ignorar
git add ~/.ssh/id_rsa # ¡NUNCA!

```

17.3 3. Commits y Mensajes

DO:

```

# Commits descriptivos y atómicos
git commit -m "feat(nvim): Add LSP configuration for Rust"
git commit -m "fix(zsh): Correct path to starship prompt"

# Un cambio lógico por commit

```

DON'T:

```

# Commits genéricos
git commit -m "Update stuff"
git commit -m "Changes"

# Múltiples cambios no relacionados en un commit

```

17.4 4. Testing Antes de Commit

DO:

```
# Siempre test antes de commit
stow -D nvim # Desinstalar
stow nvim    # Reinstalar
# Verificar que funciona
git commit

# Dry run en nueva máquina
stow -nv */
```

DON'T:

```
# Commit sin probar
# Cambios → commit → push → Rompe en otra máquina
```

17.5 5. Backup Siempre**DO:**

```
# Backup antes de stow en nueva máquina
mkdir ~/backup
cp -r ~/.config/nvim ~/backup/

# Luego stow
stow nvim
```

DON'T:

```
# Stow directamente sin backup
stow nvim # Puede sobrescribir configs importantes
```

17.6 6. Documentación**DO:**

```
# README.md completo
# - Qué paquetes hay
# - Cómo instalar
# - Dependencias
# - Comandos útiles

# Comentarios en configs
# nvim/init.lua
-- LSP configuration
-- Requires: nvim-lspconfig plugin
```

DON'T:


```
# README vacío o sin info
# Configs sin comentarios
```

17.7 7. Estructura Consistente

DO:

```
# Misma estructura en todos los paquetes
package/
  .stow-local-ignore
  README.md
  (archivos que van en HOME)
```

DON'T:

```
# Estructura inconsistente entre paquetes
```

17.8 8. Versionado

DO:

```
# Tags para versiones estables
git tag -a v1.0.0 -m "Stable nvim config"

# Branches para experimentar
git checkout -b experiment/new-theme
```

DON'T:

```
# Todo en main sin tags
# Experimentar directamente en main
```

18 Alternativas a Stow

18.1 1. yadm (Yet Another Dotfiles Manager)

Ventajas:

- Git nativo, no symlinks
- Encriptación built-in
- Templates con Jinja2
- Bootstrap scripts

Desventajas:

- Menos control granular
- Todo en un repo

```
# Instalar
sudo apt install yadm

# Usar
yadm init
yadm add ~/.zshrc
yadm commit -m "Add zshrc"
```

18.2 2. chezmoi

Ventajas:

- Templates
- Secrets management
- Cross-platform
- Estado vs archivos

Desventajas:

- Más complejo
- Curva de aprendizaje

```
# Instalar
sh -c "$(curl -fsLS get.chezmoi.io)"

# Usar
chezmoi init
chezmoi add ~/.zshrc
```

18.3 3. dotbot

Ventajas:

- Basado en configuración YAML
- Bootstrapping automático
- Plugins

Desventajas:

- Otra herramienta que aprender
- Menos flexibilidad que Stow

```
# install.conf.yaml
- link:
  ~/.zshrc: zshrc
  ~/.config/nvim: nvim
```

18.4 4. Bare Git Repository

Ventajas:

- Solo Git, no tools extra
- Total control

Desventajas:

- Más manual
- Conflictos con .gitignore

```
# Setup
git init --bare $HOME/.dotfiles
alias config='/usr/bin/git --git-dir=$HOME/.dotfiles/ --work-tree=$HOME'
config config --local status.showUntrackedFiles no

# Usar
config add .zshrc
config commit -m "Add zshrc"
```

18.5 Comparación

Feature	Stow	yadm	chezmoi	dotbot	Bare Git
Simplicidad					
Flexibilidad					
Templates					
Secrets					
Instalación	Easy	Easy	Easy	Easy	None needed
Comunidad	Grande	Media	Grande	Pequeña	N/A

Recomendación: Stow es ideal si quieres: - Simplicidad - Control total - Organización por paquetes - Solo symlinks, sin magia

19 Conclusión

19.1 Resumen de Stow

Ventajas:

- Simple y directo
- Solo hace una cosa: symlinks
- Organización clara por paquetes
- No modifica archivos originales
- Fácil de entender y debuggear
- Perfecto para dotfiles

Desventajas:

- Sin templates
- Sin manejo de secrets
- Requiere estructura específica
- Puede ser confuso al principio

19.2 Comandos Esenciales

```
# Instalar
stow paquete

# Desinstalar
stow -D paquete

# Reinstalar
stow -R paquete

# Simular
stow -nv paquete

# Ver qué hace
stow -vv paquete

# Ignorar archivos
stow --ignore='patrón' paquete

# Especificar directorios
stow -d ~/dotfiles -t ~ paquete
```

19.3 Próximos Pasos

1. **Setup inicial:** Crear estructura de dotfiles
2. **Migrar configs:** Mover configs existentes a paquetes
3. **Stow todo:** Instalar con stow
4. **Git:** Versionar con Git
5. **Probar:** En máquina de prueba o VM
6. **Iterar:** Mejorar organización según necesites

19.4 Recursos Adicionales

Documentación:

- Manual oficial: `man stow`
- Info pages: `info stow`
- Web: <https://www.gnu.org/software/stow/>

Comunidad:

- r/unixporn (ejemplos de dotfiles)
- GitHub topic: dotfiles
- YouTube: “dotfiles management”

Ejemplos de dotfiles con Stow:

- <https://github.com/search?q=stow+dotfiles>
- <https://dotfiles.github.io/>

20 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1.  [Comandos De Informacion Windows](#)
2.  [Adb](#)
3.  [Limpieza Y Optimizacion De Pc](#)
4.  [Usando Apk En Window 11](#)
5.  [Gestionar Versiones De Jdk En Kubuntu](#)
6.  [Instalar Tor Browser](#)
7.  [Crear Enlaces Duros O Hard Link En Linux](#)
8.  [Comandos Vim](#)
9.  [Guia De Git Y Github](#)
10.  [00 Primeros Pasos En Linux](#)
11.  [01 Introduccion Linux](#)
12.  [02 Distribuciones Linux](#)
13.  [03 Instalacion Linux](#)
14.  [04 Administracion Particiones Volúmenes](#)
15.  [Atajos De Teclado Y Comandos Para Usar Vim](#)
16.  [Instalando Specitify](#)
17.  [Gestiona Tus Dotfiles Con Gnu Stow](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!